

Pipelined RV32IM Core Optimization and Initial Softmax Accelerator Implementation

ELEC 5803

Jesse Levine (101185127)

1 CPU Core Optimization

1.1 Architectural Modifications

The baseline RV32IM core was optimized through the introduction of loop-level pipelining in the main program execution loop. This improvement allowed the instruction execution to overlap across cycles, reducing structural stalls and improving overall throughput. No functional changes were made to the ISA implementation; all RV32I and RV32M functionality was preserved.

The memory interface remains a unified single-port BRAM, and the architecture continues to operate as a scalar, in-order processor. The improvement is purely microarchitectural (pipeline scheduling), not algorithmic.

After pipelining, the CPI dropped to $CPI \approx 2$ from $CPI \approx 5$.

1.2 Implementation Results

The implementation is targeting the PYNQ-Z1 FPGA.

Table 1: Post-Implementation Resource Utilization

Resource	Usage
SLICE	844
LUT	2169
FF	1465
DSP	12

1.3 Software SoftMax Benchmark Results

The softmax kernel was executed entirely in software on the pipelined RV32IM core.

Empirically, the latency scales linearly:

$$T_{CPU}(N) \approx 288N + 74$$

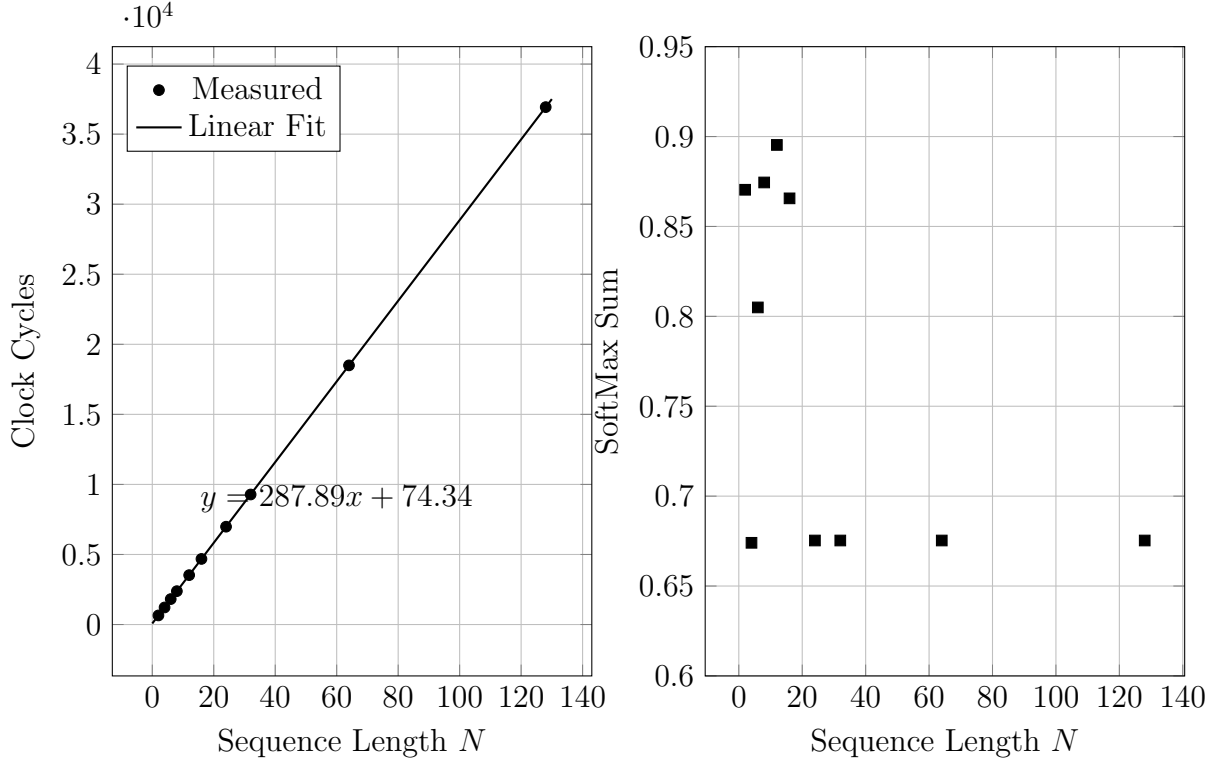


Figure 1: (a): Clock cycle scaling of pipelined core with linear regression. (b): SoftMax output sum versus sequence length N .

Corresponding to approximately 288 cycles per element, linear $O(N)$ behaviour and instruction bound execution. From Figure 1(a), for "large" $N(128)$, the total latency reached 36,928 clock cycles, establishing the performance baseline for hardware comparison.

Furthermore, as the sequence length increases, the sum degrades....

2 SoftMax Hardware Accelerator

2.1 Current MicroArchitecture Design

A dedicated SoftMax accelerator was implemented as a separate HLS IP block and integrated as a memory-mapped coprocessor alongside the RV32IM core. The accelerator exposes an AXI4 master interface for reading input logits and writing output probabilities directly to system memory, and an AXI-Lite control interface for configuration and execution control.

The CPU programs the accelerator by writing the input base address, output base address, debug base address, and vector length N into memory-mapped registers, then asserting `ap_start`. Upon completion, the accelerator raises `ap_done`, allowing the CPU to resume execution.

This architecture removes instruction-level execution of the SoftMax kernel and replaces it with a streaming dataflow engine operating directly on memory-resident data.

Unlike the software implementation, where exponentiation and normalization are computed through long sequences of scalar instructions using a Taylor series approximation, the accelerator replaces these operations with fixed-latency hardware approximations and a fully pipelined datapath.

2.1.1 Exponentiation

The exponential function is computed using:

$$e^x = 2^{x/\ln 2}$$

The input is first multiplied by the fixed-point constant $\frac{1}{\ln 2}$ in Q16.16 format. The result is decomposed into integer and fractional components. The fractional term is approximated using a 16-segment piecewise linear LUT implementing 2^{frac} , followed by linear interpolation between adjacent LUT entries. The integer component is applied through a power-of-two shift operation.

This approach replaces the multi-cycle Taylor series expansion used in software with a bounded-latency hardware implementation.

2.1.2 Normalization

Normalization is performed by first accumulating the exponentiated terms and extracting the most significant bit (MSB) to scale the dynamic range. The reciprocal of the normalized sum is then approximated using a 16-segment piecewise linear LUT implementing $\frac{1}{x}$, and a final fixed-point multiplication produces the normalized probabilities.

This removes iterative division and shift-based normalization from the critical path and converts normalization into a deterministic hardware operation.

2.1.3 Datapath Characteristics

All major computation loops are fully pipelined with an initiation interval (II) of one, resulting in one exponentiation result and one normalized probability produced per cycle after pipeline fill. The architecture therefore exhibits linear $O(N)$ throughput scaling, with execution time dominated by per-element processing rather than instruction overhead.

By eliminating instruction fetch, decode, and sequential control flow, the accelerator operates as a throughput-optimized streaming datapath engine rather than an instruction-driven processor.

2.2 Implementation

Table 2: Post-Implementation Resource Utilization

Resource	Usage
SLICE	1230
LUT	3091
FF	3624
DSP	7
BRAM	2
SRL	164

2.3 SoftMax Benchmark Results

Measured accelerator latency from rising edge of `ap_start` to rising edge of `ap_done`.

Empirically, the latency scales with the following relation:

$$T_{ACC}(N) = 4.68 + 74.17$$

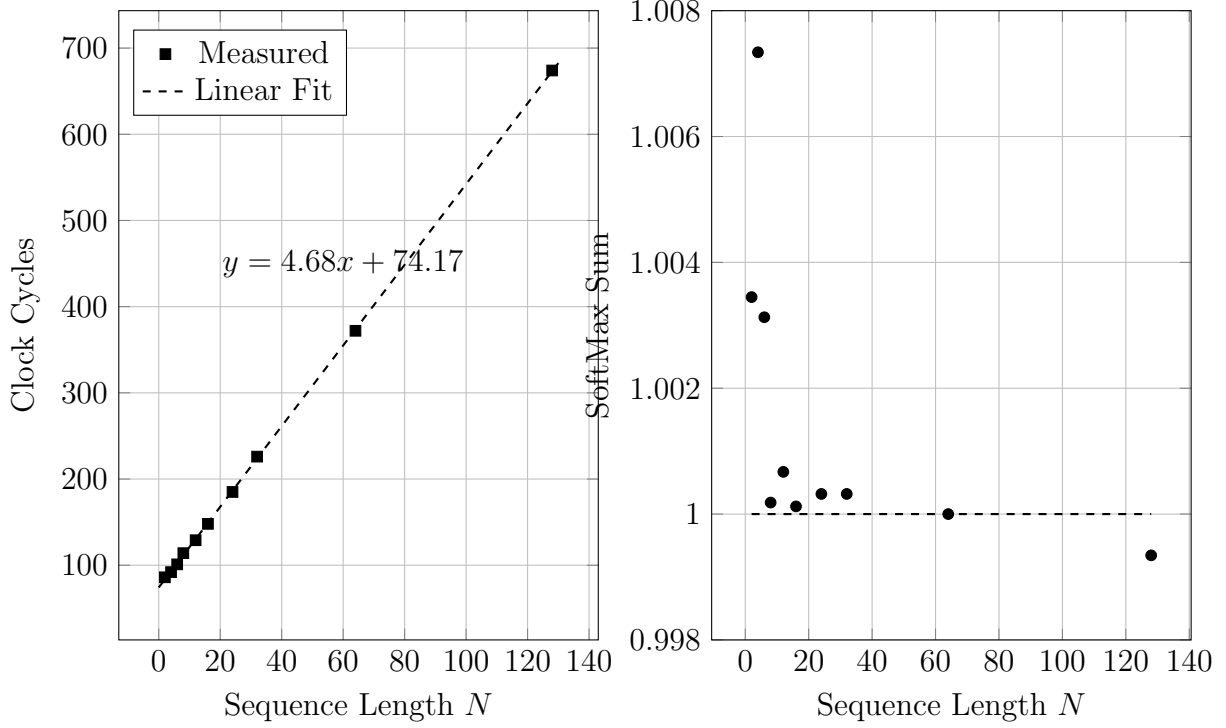


Figure 2: (a): Clock cycle scaling for SoftMax Accelerator. (b): SoftMax output sum versus sequence length N .

Thus: this results in roughly 4-5 cycles per element, linear $O(N)$ scaling and fully throughput dominated behaviour.

2.4 Performance Comparison

Speedup, in this case, is defined as:

$$Speedup(N) = \frac{T_{CPU}(C)}{T_{ACC}(N)}$$

The speedup increases with N because the CPU execution cost per element is high (≈ 288) cycles/element (Figure 1(a)) while the accelerator cost per element is constant and small ($\approx 4 - 5$) cycles/element (Figure 2(a)). Both architectures scale linearly, but with vastly different slopes.

It can be seen that as $N \rightarrow \infty$:

$$Speedup \rightarrow \frac{287.89}{4.68} \approx 61.51$$

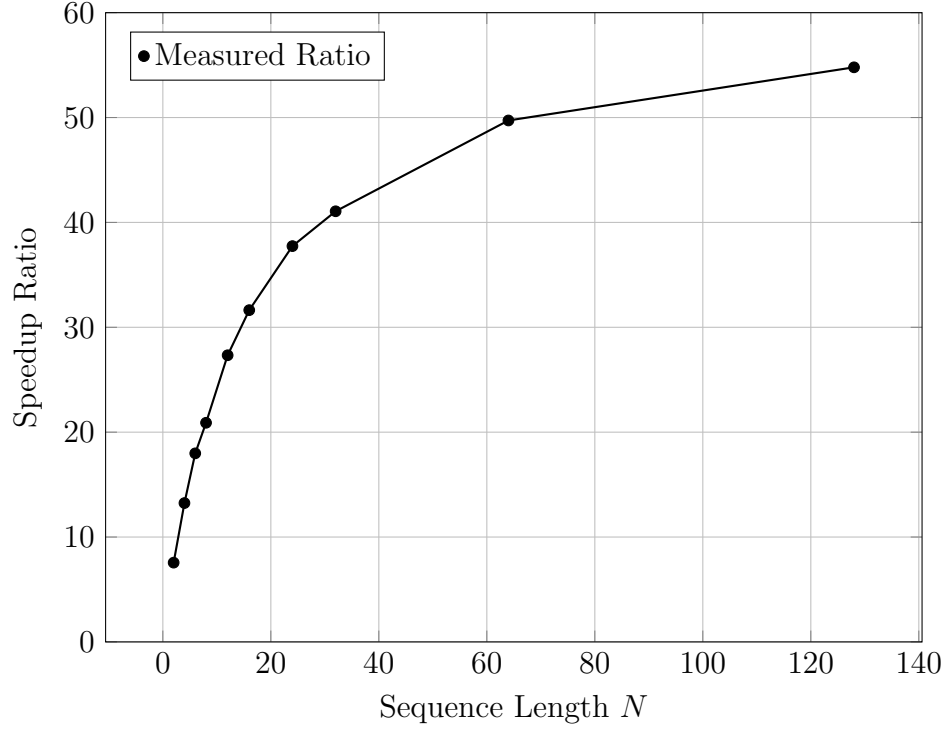


Figure 3: Speedup Ratio versus Sequence Length N .

Meaning: the performance gain is not constant-it asymptotically approaches the ratio of per-element costs, As shown in Figure 3.

2.4.1 Numerical Behaviour Comparison

Furthermore, from Figures 1(b) and 2(b), we can evaluate numerical stability. SoftMax probability sums were measured for increasing vector sizes N . For the Accelerator, Figure 2(b), across all tested sizes, the accelerator maintains:

$$\sum_i p_i \approx 1.0$$

The maximum deviation observed was $< 0.7\%$ which is at large N ($N=128$). At large N , the deviation arises from fixed-point quantization (Q16.16 format). It was observed that small probabilities eventually saturate at:

$$2^{-14} \approx 0.000061$$

which represents the minimum representable non-zero probability in Q16.16.

For the accelerator, this confirms proper max-subtraction stabilization, correct reciprocal-based normalization and numerically well conditioned exponent scaling. The small probability saturation reflects fixed-point precision limits rather than architectural error.

Regarding the CPU: the software implementation exhibits significant deviation from unity for larger N, with the sum stabilizing near:

$$\sum_i p_i \approx 0.675$$

This indicates accumulated precision loss in the Taylor series exponent approximation, instability in shift-based normalization and progressive probability collapse as dynamic range increases; unlike the accelerator, the software kernel does not preserve probabilistic normalization under increasing vector size.

All of this highlights that the hardware accelerator not only improves latency (a limit of 61.5x), but also significantly improves numerical robustness.

3 AI Usage

At this point I have fully transitioned back to pure ChatGPT usage...Sam Altman take my money! ChatGPT has helped me write the code for both the accelerator IP block and its testbench and also interpret these results.

References